

TEMA 3

CSS

Módulo: **Lenguajes de Marcas y Sistemas de Gestión de la Información**

Ciclo: **DAW**

ÍNDICE

ÍNDICE	2
INTRODUCCIÓN	4
CSS3	4
HTML5 Y CSS3	4
RETROCOMPATIBILIDAD	5
COMBINANDO CSS3 Y HTML5	6
CSS EN LÍNEA (INLINE CSS)	7
CSS INTERNO (INTERNAL CSS)	7
CSS EXTERNO (EXTERNAL CSS)	8
SELECTORES	10
JERARQUÍA DE LOS ELEMENTOS HTML	10
SELECTORES SIMPLES	12
CLASES MÚLTIPLES	13
IDENTIFICADORES MÚLTIPLES	14
SELECTOR UNIVERSAL	14
COMBINADORES	15
COLOR: RED;	17
SELECTORES DE ATRIBUTOS	17
PSEUDOCLASSES	19
CASCADA, ESPECIFICIDAD Y HERENCIA EN CSS	21
ESPECIFICIDAD	21

ESTILOS EN LÍNEA (INLINE STYLES)	22
LA PALABRA CLAVE ! IMPORTANT	22
ORDEN DE LAS REGLAS	23
HERENCIA	23
MODELO DE CAJA	24
CONTENT-BOX VS BORDER-BOX	24
EL RESET UNIVERSAL	25

Introducción

El lenguaje CSS fue creado con el objeto de separar la estructura y el contenido de un documento HTML de su presentación, esto es, el aspecto visual. Esta separación permite mejorar la accesibilidad al contenido, proporcionando mayor flexibilidad y control en la especificación de características de presentación.

La principal ventaja de CSS frente al formato HTML es que posibilita que varias páginas compartan la misma apariencia, reduciendo la complejidad y repetición del contenido estructural.

CSS3

Al contrario de sus predecesores, CSS (presentado en 1996) y CSS2 (presentado en 1998), CSS3 constituye un conjunto de módulos independientes que son publicados conforme van estando listos para el uso público, lo que hace de ella una tecnología altamente escalable y adaptable a las necesidades del mercado de la Web actual.

Hay que tener en consideración que, por el hecho de que un módulo CSS3 esté aún en el nivel de borrador no por eso no va a estar implementado en los navegadores, de hecho, es muy habitual que las sucesivas actualizaciones de los *browsers* vayan incluyendo estos módulos «provisoriales» para ir asentando su uso y adaptarse despacio a nuevos estándares. De esta forma, los diseñadores pueden ir probando noticias características de la tecnología y, así, los desarrolladores aprovechan la retroalimentación que de estos y de los navegadores les va llegando. El problema surge cuándo cada motor de cada navegador interpreta esas propiedades de manera distinta. Con la idea de resolver eso nacieron los prefijos específicos de los navegadores, de los que hablaremos en detalle más adelante.

HTML5 y CSS3

La introducción de la tecnología HTML5 (entendiendo como tal la integración de HTML5, CSS3 y JavaScript) en la Web supuso una revolución en tanto que el traslado de gran parte de la carga de trabajo del lado del cliente permitió la creación de páginas web más interactivas y complejas visualmente, con la consecuente ganancia en amigabilidad de las mismas.

Así pues, son las nuevas propiedades introducidas por CSS3 las que, principalmente, hacen a la Web actual como es: transformaciones, transiciones, animaciones, etc.

Retrocompatibilidad

Como es obvio, y considerando como dijimos que se va desarrollando y actualizando CSS3, las versiones más antiguas de los navegadores nunca van a tener soporte para las propiedades más recientes. Con el motivo de resolver esto, los programadores Web combinan una característica de HTML desenvuelta por Microsoft como son los «comentarios condicionales» (permiten tomar decisiones en función del tipo de navegador empleado) con código JavaScript que auto detecte el navegador empleado, para así escoger la hoja de estilos más idónea la cada caso. Esto implica, lógicamente, desarrollar ficheros de estilos separados para distintas versiones de un mismo navegador, con la carga de trabajo añadida que eso implica.

```
<!--[if IE 6]>
<link href="css/IE6.css" rel="stylesheet" type="text/css">
<![endif] -->
<!--[if IE 7]>
<link href="css/IE7.css" rel="stylesheet" type="text/css">
<![endif] -->
<!--[if IE 8]>
<link href="css/IE8.css" rel="stylesheet" type="text/css">
<![endif] -->
<!--[if lte IE 8]>
<link href="css/otro.css" rel="stylesheet" type="text/css">
<![endif] -->
```

Ejemplo de código CSS para navegadores Microsoft en el que se escogen distintas hojas de estilo segun la versión de Internet Explorer empleada por el usuario.

Si no tenemos que asegurar soporte a navegadores espacíficos, el problema de las leves diferencias entre navegadores al tiempo de interpretar CSS puede ser resuelto mediante los prefijos específicos de los que hablamos anteriormente.

```
.imagen {
  -webkit-transform: rotate(60deg);
  -moz-transform: rotate(60deg);
  -ms-transform: rotate(60deg);
  -el-transform: rotate(60deg);
  -khtml-transform: rotate(60deg);
  transform: rotate(60deg);
}
```

Ejemplo de uso de los prefijos específicos de CSS para distintos navegadores.

En el ejemplo anterior, podemos ver que se introduce aparentemente por sextuplicada la misma propiedad. En este caso estaríamos reflejando lo siguiente:

- **webkit:** referido a navegadores basados en el motor wekbit, tales como Safari, Google Chrome o la mayoría de navegadores para dispositivos Android.
- **moz:** referido a navegadores basados en el motor Gecko tales como Firefox o IceWeasel.
- **ms:** referido a navegadores Microsoft basados en el motor de Internet Explorer, como el propio IE o Microsoft Edge.
- **lo:** referido a los navegadores Opera.
- **khtml:** referido a Konqueror y a las versiones antiguas de Safari.

En caso de que un navegador soporte la propiedad estándar, aplicará únicamente esta. Es por ese motivo que añadimos, al final, la propiedad sin ningún tipo de prefijo.

Combinando CSS3 y HTML5

Aunque el código CSS puede anidarse en el código HTML empleando el atributo `style` (puede ser escrito dentro de él código html), tal manera de programar viola la separación entre vista y estructura de la información que persigue la propia existencia de dos tecnologías separadas, por lo que deberemos siempre evitarlo en la medida del posible.

Así, cuando nos queramos referir a documentos de estilos CSS desde un documento HTML, de forma que se encarguen de asignar la vista de la información, tendremos que añadir una línea como la que figura a continuación entre las etiquetas `<head>` y `</head>` de HTML (es decir, en la cabecera del documento) para cada uno de nuestros ficheros CSS.

```
<link rel="stylesheet" href="estilos.css" type="text/css" />
```

Donde **estilos.css** es uno de los documentos CSS que empleará nuestro documento HTML.

Desde que se presentó el estándar HTML5 el atributo *type* ya no es preciso, por lo que podríamos escribir la línea como sigue, conservando la funcionalidad:

```
<link rel="stylesheet" href="estilos.css" />
```

CSS en línea (Inline CSS)

Se aplica directamente a un elemento HTML específico, usando el atributo `style`:

```
<!DOCTYPE html>
<html>

<head>
|   <title>CSS inline</title>
</head>

<body>
|   <h1 style="color: blue; text-align: center;">Título Principal</h1>
|   <p style="font-size: 16px;">Este es un párrafo con estilo.</p>
</body>

</html>
```

Título Principal

Este es un párrafo con estilo.

CSS interno (Internal CSS)

Se define el CSS dentro de la etiqueta `<style>` en la sección `<head>` del documento HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS internal</title>
  <style>
    h1 {
      color: green;
      text-align: left;
    }
    p {
      font-size: 14px;
    }
  </style>
</head>
<body>
  <h1>Este es otro título</h1>
  <p>Este es otro párrafo con estilo diferente.</p>
</body>
</html>
```

Este es otro título

Este es otro párrafo con estilo diferente.

CSS externo (External CSS)

Se define el CSS en un archivo separado (extensión .css) y se enlaza este archivo al documento HTML mediante la etiqueta <link> en la sección <head>


```
<!DOCTYPE html>
<html>
<head>
  <title>CSS external</title>
  <link rel="stylesheet" href="estilos.css"> <!-- Enlace al archivo CSS -->
</head>
<body>
  <h1>Título con estilo externo</h1>
  <p>Párrafo con estilo externo.</p>
</body>
</html>
```

Título con estilo externo

Párrafo con estilo externo.

Como norma general, se deben usar **CSS externos** siempre que sea posible, ya que en la mayoría de proyectos lo que queremos es utilizar los mismos estilos a todo el proyecto y de este modo tendremos un código más limpio ya que no incrustamos código repetido en las diferentes páginas.

El **CSS interno** se suele usar para estilos específicos de una sola página, excepciones dentro del proyecto que no vamos a reutilizar.

El **CSS en línea** se usa para modificaciones temporales o casos muy excepcionales, en los que necesitemos anular estilos que se estén aplicando. No se deben usar de forma generalizada.

Selectores

Un **selector** o estilo ¹ en CSS es una etiqueta que permite identificar aquel elemento o conjunto de elementos HTML a los que va a ser aplicado. Su estructura consta de dos partes:

- **Selector:** etiqueta de texto que determina el elemento HTML (o conjunto de elementos) a los que es aplicable el estilo.
- **Declaración:** define las propiedades y valores que constituyen el estilo marcado por la estructura. Ve siempre seguida del selector y acotada por llaves (**{}**).

Jerarquía de los elementos HTML

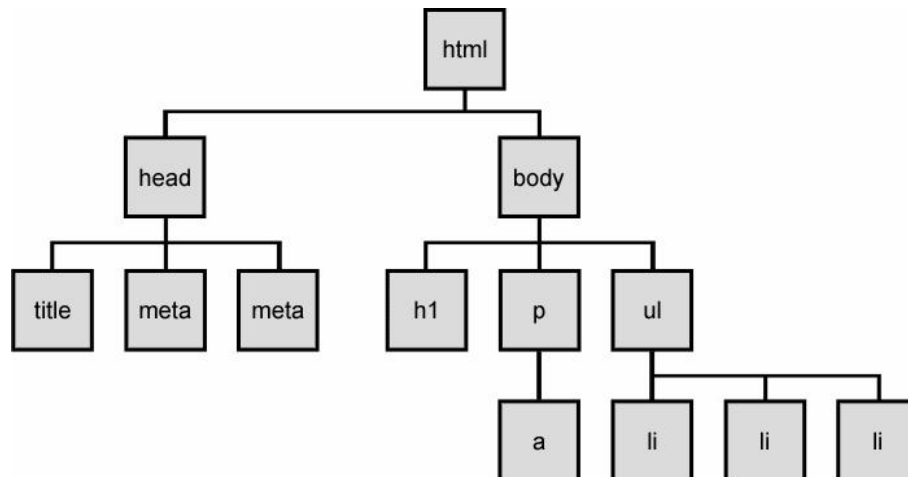
Para poder manejar de manera apropiada los selectores de CSS es preciso conocer la jerarquía de los elementos que van a componer una página HTML. Destacaremos aquí sólo los de uso habitual, e iremos introduciendo el resto a lo largo del tema a través de ejemplos prácticos.

Para un ejemplo como lo que sigue:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Ejemplo para la jerarquía de elementos HTML</title>
    <meta http-equiv="content-type" content="text/html;
charset=utf-8" />
    <meta http-equiv="Content-Language" content="eres" />
  </head>
  <body>
    <h1>Esto es una cabecera de texto</h1>
    <p>Un párrafo con una
      <a href="http://www.google.com/">enlace</a>.
    </p>
    <ul>
      <li>Un elemento de lista.</li>
      <li>Otro.</li>
      <li>Más uno.</li>
    </ul>
  </body>
</html>
```

¹ Coloquialmente es habitual verlos denominados como **estilos**, aunque la designación oficial sea **selectores**.

Así pues, en un código como el anterior, los elementos HTML de la página poseen la siguiente jerarquía:



Tenemos, por lo tanto, que introducir una serie de conceptos relativos a las relaciones jerárquicas entre elementos HTML que nos serán de ayuda al tiempo de trabajar con CSS:

- **Sucesor** o descendiente: elemento que está situado en el sub-árbol de un elemento concreto (es decir, a lo que se llega sólo por caminos descendientes). Por ejemplo: *para el elemento **body** serían sucesores **h1**, **p**, **li**, **ul** y todos los **li**.*
- **Antecesor** o ascendente: elemento que está situado en el super-árbol de un elemento concreto (esto es, a lo que se llega sólo por caminos ascendentes). Por ejemplo: *el elemento **body** tendría como único antecesor a **html**.*
- **Hijo**: elemento que es sucesor directo de un elemento concreto (es decir, sólo hay que recorrer un único tramo de camino descendiente para llegar a él). Por ejemplo: *para el elemento **body** serían hijos solo **h1**, **p** y **ul**.*
- **Padre**: elemento que es antecesor directo de un elemento concreto (es decir, sólo hay que recorrer un único tramo de camino ascendente para llegar a él). Por ejemplo: *para cualquiera de los elementos **li** su padre sería **ul**.*
- **Hermano**: elemento que posee el mismo elemento padre que el elemento en tela de juicio (es decir, sólo hay que recorrer un único tramo de camino ascendente seguido de un único tramo de camino descendiente para llegar a él). Por ejemplo: *serían hermanos los elementos **title** y ambos **meta**.*

Subsecuentemente, se pueden definir todo tipo de relaciones de parentesco a semejanza de aquellas de los árboles genealógicos, aunque no tienen por que ser de especial utilidad.

Selectores simples

CSS distingue tres tipos de selectores **simples** según la cantidad de elementos HTML a los que afectan y la manera en que se determinan los elementos afectados:

- **Tipo de elemento:** empleando directamente el nombre de algún *tag* de HTML conocido, de forma que sus propiedades se aplicarán a todos los elementos diera tipo con independencia de su posición en el árbol del documento. En el fragmento de código que figura a continuación vemos un ejemplo de estilo CSS aplicado a los elementos HTML de tipo párrafo (p).

```
p {  
  font-size: 2em;  
  color: blue;  
}
```

- **Identificador:** irá precedido del sostenido (#) y se aplicará a aquellos elementos² HTML que correspondan con el identificador establecido en su atributo **id** con independencia de su posición en la jerarquía del documento. El código que figura a continuación muestra la aplicación de un estilo de texto al elemento que tenga el identificador **cabecera** en el documento afectado.

```
#cabecera {  
  font-size: 14em;  
  color: black;  
}
```

- **Clase:** irá precedido del punto (.) y se aplicará sólo a aquellos elementos HTML que correspondan con la clase establecida en su atributo **class**, con independencia de su posición en el árbol del documento. En el siguiente ejemplo de código podemos ver cómo aplicar un estilo de borde a los elementos que pertenezcan a la clase **cuadro** en el documento afectado.

```
.cuadro {  
  border: 1px solid black;  
}
```

² La recomendación del estándar HTML5 es que los **id** sean únicos, esto es, que no haya varios elementos compartiendo identificador en un mismodocumento HTML, aunque no es obligatorio.

Clases múltiples

Un mismo elemento HTML puede tener asociadas varias clases (se indicarían separadas por espacios pero aplicadas igualmente desde el atributo **class**), lo que puede tener implicaciones sobre el tratamiento de las mismas dependiendo de cómo definamos los estilos correspondientes en CSS. Por ejemplo:

```
.estilo {  
  background-color: yellow;  
}  
  
p.estilo {  
  color: blue;  
}  
  
div.estilo {  
  border: 1px solid red;  
}
```

En este ejemplo, todos los elementos de la clase `estilo` tendrán fondo amarillo pero, particularmente, aquellos que sean de tipo `p` tendrán el texto en color azul, mientras que los que sean de tipo `div` tendrán un borde simple de 1 píxel de largo y color rojo.

En el caso de asignar múltiples clases a un elemento, se le aplicarán todas las características definidas por cada una de esas clases, imponiéndose las definidas posteriormente en caso de conflicto. Por ejemplo:

```
.estiloA {  
  color: gray;  
  background-color: blue;  
}  
  
.estiloB {  
  font-weight: bold;  
  background-color: red;  
}
```

En caso, un `div` al que habían sido aplicados los estilos `estiloA` y `estiloB` definidos en el ejemplo directamente superior, tendría las siguientes características: texto en color gris –según `estiloA`– y negrita –según `estiloB`– y color de fondo rojo –en tanto que `estiloB` fue definido después y sobrescribe la característica de `estiloA`. **Es decir, predomina de manera secuencial la elección de estilos. Siendo el último el que predomina sobre el primero.**

A mayores, podríamos aplicar ciertas características sólo a los elementos que pertenezcan simultáneamente a determinadas clases, de la manera:

```
.estiloA.estiloB {  
    border-color: orange;  
}
```

En el ejemplo anterior estamos implicando que la aquellos elementos que pertenezcan simultáneamente a las clases `estiloA` y `estiloB` les será aplicada un color de borde naranja.

Identificadores múltiples

De manera casi análoga a como ocurre con las clases y, como ya comentamos con anterioridad, es posible aplicar un identificador a diferentes elementos (aunque esto no se recomienda). Si, por la razón que fuese, hacemos uso de esa posibilidad, CSS nos da asimismo la posibilidad de discriminar por tipo de elemento con una sintaxis semejante a la empleada con las clases. Por ejemplo:

```
#texto_normal {  
    font-weight: bold;  
}  
  
p#texto_normal {  
    color: green;  
}
```

De la manera anterior, todos los elementos que empleen el identificador `texto_normal` tendrán formato de negrita en los textos, pero sólo aquellos de tipo `p` tendrán el texto en color verde.

Selector universal

Dada la necesidad de redefinir las características de formato que vienen preestablecidas por los navegadores para los documentos HTML (como, por ejemplo, el uso de la fuente `Times New Roman`), con CSS2 fue introducida la posibilidad de referirse a todos los elementos de un documento HTML mediante lo uso de uno único **selector universal**, tal y como vemos en el ejemplo a continuación:

```
* {  
    color: cyan;  
    font-size: 18px;  
}
```

```
}
```

Indicando el anterior implicaría que, por defecto, todos los elementos del documento HTML tendrían texto de color cian y tamaño de fuente 18 píxeles.

Entre los usos más extendidos del selector universal encontramos el conocido como *CSS reset*, en el que se redefinen todas las características que nos interese para los documentos HTML que vayan a emplear nuestra hoja de estilos. Por ejemplo:

```
* {  
  margin: 0;  
  padding: 0;  
  border: 0;  
  font-size: 100%;  
  font: inherit;  
  vertical-align: baseline;  
}
```

En ocasiones los va a interesar aplicar restricciones sólo a cierto sub-árbol de la jerarquía del documento HTML (por ejemplo, a todos los descendientes de un elemento).

```
#texto_normal * {  
  text-align: center;  
}
```

En el ejemplo anterior estaríamos indicando que todos los elementos descendientes de aquellos que tengan el identificador `texto_normal` emplearán alineamiento centrado nos sus textos.

Combinadores

Existe más una manera de aplicar características de manera generalizada a los elementos de un documento HTML al otro lado de las clases y los identificadores: los **combinadores** o selectores **combinados**. Este tipo de selectores hacen uso de cuatro caracteres distintos según la relación genealógica existente entre los elementos afectados:

- **Coma (,):** permite referirse a todos los elementos de un listado. Por ejemplo, se queremos afectar a todos los elementos `p`, `div` y además al elemento con el identificador `prueba` haremos algo como el siguiente:

```
p,div,#prueba {  
  background-color: green;
```

```
}
```

- **Espacio ():** permite referirse a todos los **elementos descendientes**. Retomando el ejemplo empleado en el punto anterior, podemos encadenar selectores universales separados por espacios para afectar la diferentes niveles de la jerarquía, por ejemplo:

```
#texto_normal * p {  
    font-size: 10px;  
} /* Estas propiedades afectarían a todos los elementos de tipo p  
nietos (e inferiores) del elemento con identificador texto_normal.  
*/  
  
body * * div {  
    color: red;  
} /* Estas propiedades afectarían a todos los elementos de tipo  
div bisnietos (e inferiores) del elemento body. */
```

En el ejemplo anterior estaríamos estableciendo dos reglas: por una parte, los elementos de tipo `p` que sean "nietos" de elementos con el identificador `texto_normal` emplearán un tamaño de fuente de 10 píxeles; entretanto que los elementos de tipo `div` que sean bisnietos del elemento `body` tendrán el texto en color rojo.

- **Mayor (>):** permite referirse a todos los **elementos hijos**, pero no así los descendientes de estos, es decir, sólo toma el primer nivel de descendientes del elemento afectado.

```
#especial > div {  
    border-color: brown;  
}
```

Considerando el ejemplo anterior tendríamos que todos los elementos de tipo `div` que sean directamente hijos de elementos que empleen el identificador `especial` tendrán borde de color marrón.

- **Suma (+):** permite referirse al **primer elemento hermano** del elemento afectado:

```
p + div {  
    color: blue;  
}
```

Segun el ejemplo anterior aquellos elementos de tipo `div` que aparezcan en el documento HTML justo después de un elemento `p` ("al lado") tendrán su texto

en color azul (si hubiese varios `div` consecutivos sólo se vería afectado el primero de ellos).

- **Tilde de la ñ o “Virgulilla” (~):** permite referirse a **todos los elementos hermanos** del elemento afectado.

```
h1 ~ h2{  
  font-weight: bold;  
}
```

Según el ejemplo anterior, todos los elementos `h2` que aparezcan después de un elemento `h1` del que sean hermanos tendrán el texto en negrita.

Todos los anteriores pueden ser combinados entre sí para implicar relaciones más complejas, como por ejemplo:

```
p > div + ul {  
  color: red;  
}
```

En el caso anterior, todos los primeros elementos `ul` que aparezcan directamente después de un hermano `div` y sean hijos de un elemento `p`, tendrán el texto en color rojo.

Selectores de atributos

Los **selectores de atributos** permiten aplicar distintos estilos CSS a los elementos de un documento HTML según los atributos que empleen e incluso, en función de sus valores. En este caso, existen siete variantes, que serían:

- **Selector de atributos simple:** se aplica a aquellos elementos HTML que hayan definido el atributo indicado, con independencia de su valor:

```
a[href], td[colspan] {  
  color: green;  
}
```

En el ejemplo anterior estaríamos poniendo el texto en color verde para todos los elementos `a` que hayan definido el atributo `href`, así como a todos los elementos `td` que estén haciendo uso de su atributo `colspan`.

- **Selector de atributos exactos:** permite aplicar estilos a elementos HTML en función del valor exacto de sus atributos.

```
img[alt="Esta imagen representa una casa"] {  
  width: 200px;
```

```
}
```

El ejemplo directamente superior aplica un largo de 200 píxeles a aquellos elementos de tipo `img` que tengan en el atributo `alt` el valor exacto *Esta imagen representa una casa*³.

Además del anterior, este selector es aplicable en negativo, indicando por ejemplo aquellos atributos que **no** cumplan la condición:

```
img:not([alt]) {  
  width: 100px;  
}
```

En este último ejemplo, el selector aplicará una largura de 100 píxeles a aquellas imágenes que no tengan valor definido en su atributo `alt`.

- **Selector de atributos por palabras:** permite aplicar estilos a elementos HTML que tengan atributos con un valor que contenga el texto indicado como palabra⁴.

```
img[src="imagen"] {  
  border: 2px dotted gray;  
}
```

El caso anterior tomará todos aquellos elementos de tipo `img` cuyo atributo `src` contenga exáctamente la cadena de texto *imagen* y les aplicará un borde de color gris punteado de 2 píxeles de largo.

- **Selector de lenguajes:** permite seleccionar los elementos HTML según el lenguaje aplicado al documento.

```
p[lang|="e"] {  
  font-family: Arial;  
}
```

Así pues, para el ejemplo anterior, se aplicará el estilo (fuente de tipo `Arial`) a aquellos elementos de tipo `p` que tengan como lenguaje definido el castellano.

³ Es importante recordar, para todos estos selectores, que **CSS distingue mayúsculas de minúsculas**.

⁴ Esto es, delimitado por espacios, tabulaciones, signos de puntuación, etc. Pero no contenido en otras palabras (en cuyo caso sería una subcadena).

- **Selector de atributos por valor de inicio:** permite aplicar estilos a elementos HTML que tengan atributos definidos cuyo valor comience con la información indicada.

```
a[href^="https://"] {  
  color: green;  
}
```

El ejemplo directamente superior aplicará color verde al texto de aquellos elementos de tipo `a` cuyo atributo `href` comience por `https://`.

- **Selector de atributos por valor de fin:** permite aplicar estilos a elementos HTML que tengan atributos definidos cuyo valor final con la información indicada.

```
img[src$=".jpg"] {  
  height: 80px;  
}
```

El estilo anterior se aplicará, pues, a aquellos elementos de tipo `img` cuyo atributo `src` final con el texto `.jpg`.

- **Selector de atributos por contenido parcial:** permite aplicar estilos a elementos HTML en función de las subcadenas que contengan los valores de sus atributos.

```
a[href*="www"] {  
  font-family: Verdana;  
}
```

El anterior ejemplo muestra un caso de aplicación de una fuente (`Verdana`) a los elementos de tipo `a` que contengan, para su atributo `href`, la subcadena de texto `www`.

Pseudoclases

Las **pseudoclases** o pseudoselectores de CSS permiten determinar el estilo de un elemento HTML basándose en metainformación externa al árbol del documento, es decir, información al otro lado de la etiqueta, los atributos (y sus valores) y el contenido.

La tabla siguiente muestra las pseudoclases disponibles en CSS3 con su sintaxis de uso y una breve descripción.

Sintaxis	Descripción
<code>etiqueta:active</code>	Se aplica al elemento cuando se está sosteniendo el clic del ratón en él.
<code>etiqueta::after</code>	Inserta algo después de cada elemento de un cierto tipo.
<code>etiqueta::before</code>	Inserta algo antes de cada elemento de un cierto tipo.
<code>input:checked</code>	Se aplica a los <code>input</code> seleccionados.
<code>input:default</code>	Se aplica a los <code>input</code> por defecto.
<code>input:disabled</code>	Se aplica a los <code>input</code> deshabilitados.
<code>etiqueta:empty</code>	Se aplica la cada etiqueta de un tipo que no tenga hijos.
<code>input:enabled</code>	Se aplica a los <code>input</code> habilitados.
<code>p:first-child</code>	Se aplica a cada <code>p</code> que es el primer hijo de su padre.
<code>p::first-letter</code>	Se aplica a la primera letra de cada <code>p</code> .
<code>p::first-line</code>	Se aplica a la primera línea de cada <code>p</code> .
<code>p:first-of-type</code>	Se aplica la cada <code>p</code> que es el primero descendiente <code>p</code> de otro.
<code>input:focus</code>	Se aplica al <code>input</code> que tiene el foco.
<code>etiqueta:hover</code>	Se aplica al elemento ubicado bajo el cursor del ratón.
<code>input:in-range</code>	Se aplica a los <code>input</code> cuyo valor esté había sido del rango indicado.
<code>input:indeterminate</code>	Se aplica a los <code>input</code> en estado indeterminado.
<code>input:invalid</code>	Se aplica a los <code>input</code> que tengan un valor inválido.
<code>etiqueta:lang(<i>idioma</i>)</code>	Se aplica la cada elemento diera tipo quet tenga el idioma indicado.
<code>etiqueta:last-child</code>	Se aplica la cada elemento que sea el último hijo de su padre.
<code>etiqueta:last-of-type</code>	Se aplica la cada elemento que sea el último elemento de un tipo específico dentro de su elemento padre
<code>a:link</code>	Se aplica la hipervínculos no visitados.
<code>:not(etiqueta)</code>	Se aplica a todos los elementos que no sean del tipo indicado.
<code>etiqueta:nth-child(n)</code>	Se aplica la cada elemento que sea el hijo n-ésimo de su padre.
<code>etiqueta:nth-last-child(n)</code>	Se aplica la cada elemento que sea hijo n-ésimo de su padre contando desde lo último hijo.
<code>etiqueta:nth-last-of-type(n)</code>	Se aplica la cada elemento que sea el n-ésimo hijo diera tipo de su padre contando desde lo último hijo.
<code>etiqueta:nth-of-type(n)</code>	Se aplica la cada elemento que sea el n-ésimo hijo de un tipo dado.
<code>etiqueta:only-of-type</code>	Se aplica la cada elemento que sea solo si es el único de su tipo (nombre de etiqueta) entre sus hermanos.
<code>etiqueta:only-child</code>	Se aplica la cada elemento que sea hijo único.
<code>input:optional</code>	Se aplica a los <code>input</code> no tengan el atributo <i>required</i> definido.
<code>input:out-of-range</code>	Se aplica a los <code>input</code> cuyo valor esté fuera del rango indicado.
<code>input::placeholder</code>	Se aplica a los <code>input</code> que tengan el atributo <i>placeholder</i> definido.
<code>input:read-only</code>	Se aplica a los <code>input</code> que tengan el atributo <i>readonly</i> definido.
<code>input:read-write</code>	Se aplica a los <code>input</code> que no tengan el atributo <i>readonly</i> definido.
<code>input:required</code>	Se aplica a los <code>input</code> tengan el atributo <i>required</i> definido.
<code>:root</code>	Se aplica al elemento <i>raíz</i> del documento.

Sintaxis	Descripción
<code>::selection</code>	Se aplica a la porción de elemento seleccionada por el usuario (por ejemplo mediante el arrastre de selección del ratón).
<code>:target</code>	Se aplica al elemento activo actual.
<code>input:valid</code>	Se aplica a todos los <code>input</code> con un valor válido.
<code>a:visited</code>	Se aplica a los hipervínculos visitados.

Cascada, especificidad y herencia en CSS

El término “en cascada” (cascading) en CSS se refiere a la forma en el que el navegador elige qué estilos aplicar a un determinado elemento cuando existen conflictos (es decir, existe más de una regla que intenta cambiar la misma propiedad del elemento).

El navegador sigue este orden de prioridad a la hora de resolver conflictos:

1. **Importancia** (si tiene la regla *!important*)
2. **Especificidad** (cual es el peso del selector, cómo de específico es)
3. **Orden de origen** (cual es, en orden del código, la posición de la regla).

Especificidad

La especificidad es el peso que tiene un selector, y se calcula mediante un sistema de puntuación de tres columnas (A, B, C):

- **(A) Identificadores (ID):** Es el peso más alto (por ejemplo `#menu`).
- **(B) Clases, Pseudoclases y Atributos:** Peso medio. (Ej: `.boton`, `:hover`, `[type="text"]`).
- **(C) Elementos y Pseudoelementos:** El peso más bajo. (Ej: `div`, `h1`, `::before`).

Como apunte, el selector universal (*) no tiene valor de especificidad, vale 0.

Ejemplo:

```
<p id="parrafo-unico" class="texto-destacado">Hola Mundo</p>
```

Si para el anterior HTML tenemos este CSS:

```
/* Puntuación: 0-0-1 */
p {
    color: black;
}

/* Puntuación: 0-1-0 */
.texto-destacado {
    color: blue;
}

/* Puntuación: 1-0-0  <-- Se aplica este */
#parrafo-unico {
    color: red;
}
```

El texto estará en rojo porque es el id más específico

Estilos en Línea (Inline Styles)

Los estilos aplicados directamente en el HTML mediante el atributo *style* tienen una especificidad superior a cualquier selector del archivo CSS (ID, clase o elemento).

```
<!-- Este color ganará a cualquier ID o clase en el CSS externo -->
<p style="color: purple;">Texto morado</p>
```

La palabra clave **!important**

Si se añade *!important* al final de una declaración, esta anulará todas las reglas anteriores, incluyendo estilos en línea.

```
p {
    color: green !important; /* Gana a todo lo demás */
}
```

Orden de las reglas

Si dos selectores tienen exactamente la misma especificidad, gana la regla que esté escrita última en orden del archivo CSS.

```
.alerta {  
    color: red;  
}  
  
/* Esta regla sobrescribe a la anterior porque está después  
   y tiene la misma especificidad (0-1-0) */  
.alerta {  
    color: orange;  
}
```

Herencia

Algunas propiedades CSS se heredan de los elementos padres a los hijos (como color, font-family, font-size), mientras que otras no (como border, margin, padding, width).

Para forzar la herencia en propiedades que no lo hacen por defecto, podemos usar el valor *inherit*.

```
div {  
    border: 1px solid black;  
}  
  
p {  
    /* El párrafo heredará el borde del div padre */  
    border: inherit;  
}
```

Modelo de caja

En CSS, la presentación de cualquier elemento se hace mediante una serie de cajas rectangulares (incluso si solo queremos representar una letra lo tomará como un rectángulo).

El modelo de cajas que utiliza es el siguiente:

- **Contenido (Content):** Es el corazón de la caja. Aquí está el elemento HTML a representar (el texto, la imagen, el video,...). Sus dimensiones se definen con width (ancho) y height (alto).
- **Relleno (Padding):** Es el espacio transparente entre el contenido y el borde.
 - Sirve para que el texto no se pegue a los bordes.
 - Toma el color de fondo (background) del elemento.
 - Podemos verlo como el cartón blanco alrededor de una foto dentro de un marco.
- **Borde (Border):** Es la línea que rodea al relleno y al contenido.
 - Puede tener grosor, estilo y color.
 - En este caso sería el marco de madera del cuadro.
- **Margen (Margin):** Es el espacio transparente fuera del borde.
 - Separa el elemento de sus vecinos.
 - No tiene color de fondo (siempre es transparente).
 - Sería la distancia entre un cuadro y otro colgado en la pared.

content-box vs border-box

Existen dos formas de calcular el tamaño de la caja, definidas por la propiedad *box-sizing*.

1. La forma antigua (y forma por defecto): *box-sizing: content-box*

En este caso cuando se aplica por ejemplo un ancho (width:200px), CSS aplica ese ancho tan solo al content. En el caso de aplicar además un padding o un border, estos se suman a ese ancho.

Veamos con un ejemplo:

```
.forma-antigua {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid black;  
  box-sizing: content-box; /* Valor por defecto */  
}
```

Tenemos esta salida:



Si miramos el ancho de esa div, tendríamos que:

200px (ancho) + 40px (padding izq + der) + 10px (borde izq + der) = 250px.

2. La forma moderna (box-sizing: border-box)

```
.forma-moderna {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid black;  
  box-sizing: border-box; /* La forma recomendada */  
}
```

En este caso, al decirle un ancho de 200px, aplicará ese ancho al conjunto, incluyendo el padding o el border.



El reset universal

En el caso de no querer lidiar con las matemáticas del content-box, se suele indicar en el fichero css un reset para utilizar el modelo border-box:

```
/* Reset Universal */  
* {  
  box-sizing: border-box;  
}
```

```
/* Opcional pero recomendado para herencia */  
*, *::before, *::after {  
  box-sizing: border-box;  
}
```